

```
=====
BIBLE MYSQL · PSYCOPG2 · PYMYSQL – RÉFÉRENCE EXHAUSTIVE
LÉGENDE : [pg] = psycopg2 (PostgreSQL) | [my] = PyMySQL / mysql-connector
          [sql] = SQL commun | [py] = Python
INSTALL : pip install psycopg2-binary pymysql mysql-connector-python
=====
```

[pg] psycopg2 – Connexion PostgreSQL

DESCRIPTION : Adaptateur PostgreSQL le plus utilisé en Python (DB-API 2.0)
 POURQUOI : Performances, types PostgreSQL natifs, async, pooling
 CONTEXTE : Applications web, data pipelines, systèmes transactionnels

— CONNEXION —

```
import psycopg2
import psycopg2.extras
import psycopg2.pool

# Connexion basique
conn = psycopg2.connect(
    host="localhost",
    port=5432,
    database="mabase",
    user="utilisateur",
    password="motdepasse",
)

conn = psycopg2.connect("host=localhost port=5432 dbname=mabase user=user password=pass")
conn = psycopg2.connect("postgresql://user:pass@localhost:5432/mabase")
conn = psycopg2.connect("postgresql://user:pass@localhost/mabase?sslmode=require")
conn = psycopg2.connect(dsn="postgresql://user:pass@host/db")

# Paramètres supplémentaires
conn = psycopg2.connect(
    host="localhost",
    database="mabase",
    user="user",
    password="pass",
    connect_timeout=10,
    application_name="mon_app",
    options="-c search_path=monschema",
    sslmode="require",
    sslcert="/chemin/client.crt",
    sslkey="/chemin/client.key",
    sslrootcert="/chemin/ca.crt",
    keepalives=1,
    keepalives_idle=30,
    keepalives_interval=10,
    keepalives_count=5,
)

# Fermeture
conn.close()

# Context manager (autoclose)
with psycopg2.connect(...) as conn:
    # NB : ne ferme PAS conn à la sortie (commit/rollback seulement)
    pass
conn.close()
```

Timeout connexion (secondes)
 Visible dans pg_stat_activity
 Options de session
 require, verify-full, disable...

Activer TCP keepalive

Fermeture explicite nécessaire

— PROPRIÉTÉS CONNEXION —

conn.closed	0 = ouverte, >0 = fermée
conn.status	STATUS_READY, STATUS_BEGIN...
conn.info.server_version	Version serveur (ex: 150003)
conn.info.protocol_version	Version protocole
conn.info.backend_pid	PID du process backend
conn.info.dbname	Nom de la base
conn.info.user	Utilisateur connecté
conn.info.host	Hôte
conn.info.port	Port
conn.info.password	Mot de passe
conn.info.ssl_in_use	True si SSL actif
conn.info.ssl_attribute('protocol')	Attribut SSL spécifique
conn.get_dsn_parameters()	Dict des paramètres DSN
conn.get_parameter_status('server_version')	Paramètre de session PostgreSQL

— TRANSACTION —	
<code>conn.commit()</code>	Valider la transaction
<code>conn.rollback()</code>	Annuler la transaction
<code>conn.autocommit = True</code>	Mode autocommit (pas de transaction)
<code>conn.autocommit = False</code>	Mode transactionnel (défaut)
<code>conn.set_isolation_level(level)</code>	Niveau d'isolation
<code>conn.set_session(isolation_level='READ COMMITTED')</code>	
<code>conn.set_session(isolation_level='REPEATABLE READ')</code>	
<code>conn.set_session(isolation_level='SERIALIZABLE')</code>	
<code>conn.set_session(readonly=True)</code>	Session en lecture seule
<code>conn.set_session(deferrable=True)</code>	Transaction différable
<code>conn.set_session(autocommit=True)</code>	Autocommit
<code>import psycopg2.extensions as ext</code>	
<code>ext.ISOLATION_LEVEL_AUTOCOMMIT</code>	= 0
<code>ext.ISOLATION_LEVEL_READ_UNCOMMITTED</code>	= 4
<code>ext.ISOLATION_LEVEL_READ_COMMITTED</code>	= 1 (défaut PostgreSQL)
<code>ext.ISOLATION_LEVEL_REPEATABLE_READ</code>	= 2
<code>ext.ISOLATION_LEVEL_SERIALIZABLE</code>	= 3
<code>ext.ISOLATION_LEVEL_DEFAULT</code>	= None
— CURSEUR —	
<code>cur = conn.cursor()</code>	Curseur standard (tuples)
<code>cur = conn.cursor(name='mon_curseur')</code>	Curseur côté serveur (streaming)
# Curseurs spéciaux	
<code>cur = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)</code>	dict
<code>cur = conn.cursor(cursor_factory=psycopg2.extras.DictCursor)</code>	dict-like
<code>cur = conn.cursor(cursor_factory=psycopg2.extras.NamedTupleCursor)</code>	namedtuple
<code>cur = conn.cursor(cursor_factory=psycopg2.extras.LoggingCursor)</code>	avec log
<code>cur.close()</code>	Fermer le curseur
<code>cur.closed</code>	True si fermé
# Propriétés du curseur	
<code>cur.description</code>	Métadonnées colonnes (après SELECT)
<code>cur.rowcount</code>	Nb lignes affectées / retournées
<code>cur.statusmessage</code>	Message statut dernière commande
<code>cur.query</code>	Dernière requête envoyée (bytes)
<code>cur.connection</code>	Connexion parente
<code>cur.name</code>	Nom (curseur serveur)
<code>cur.scrollable</code>	Si le curseur est scrollable
<code>cur.withhold</code>	Si maintenu après COMMIT
<code>cur.itsize</code>	Taille des lots (curseur serveur, défaut 2000)
<code>cur.arraysize</code>	Taille fetch (défaut 1)
<code>cur.rownumber</code>	Index ligne courante
<code>cur.lastrowid</code>	OID de la dernière ligne insérée
— EXÉCUTION —	
<code>cur.execute(sql)</code>	Exécuter une requête
<code>cur.execute(sql, params)</code>	Avec paramètres (sécurisé)
<code>cur.execute("SELECT %s, %s", (1, 'texte'))</code>	Paramètres positionnels
<code>cur.execute("SELECT %(nom)s", {'nom': 'val'})</code>	Paramètres nommés
<code>cur.execute("INSERT INTO t VALUES %s", (val,))</code>	ATTENTION : virgule pour tuple de 1
<code>cur.executemany(sql, liste_params)</code>	Exécuter plusieurs fois
<code>cur.executemany("INSERT INTO t VALUES (%s,%s)", [(1,'a'),(2,'b')])</code>	
<code>cur.callproc('ma_procedure', [arg1, arg2])</code>	Appeler une procédure stockée
<code>cur.callproc('ma_func', [arg1, arg2])</code>	
<code>cur.fetchone()</code>	Résultats après callproc
# IMPORTANT : JAMAIS de f-string ou % pour les paramètres SQL (injection)	
<code># x cur.execute(f"SELECT * FROM t WHERE id = {id}")</code>	
<code># ✓ cur.execute("SELECT * FROM t WHERE id = %s", (id,))</code>	
— FETCH —	
<code>cur.fetchone()</code>	Une ligne (tuple) ou None
<code>cur.fetchmany(size=10)</code>	N lignes (liste de tuples)
<code>cur.fetchall()</code>	Toutes les lignes
<code>cur.fetchmany(cur.arraysize)</code>	Selon arraysize
# Itération	
<code>cur.execute("SELECT * FROM ma_table")</code>	

```

for ligne in cur:
    print(ligne)
                                Itération directe (efficace)

# Scroll (curseur serveur)
cur.scroll(5)                    Avancer de 5
cur.scroll(0, mode='absolute')   Aller à la position 0
cur.scroll(-1, mode='relative')  Reculer de 1

— CURSEUR SERVEUR (streaming) —
# Pour grandes requêtes : résultats en streaming (sans charger en RAM)
with conn.cursor('curseur_stream') as cur:
    cur.itsize = 1000             Lot de 1000 lignes à la fois
    cur.execute("SELECT * FROM grande_table")
    for ligne in cur:
        traiter(ligne)

# Avec HOLD (survit au COMMIT)
with conn.cursor('curseur_hold', withhold=True) as cur:
    cur.execute("SELECT * FROM t")
    conn.commit()
    for ligne in cur:
        traiter(ligne)

— EXTRAS : OPÉRATIONS BATCH —
from psycpg2.extras import execute_values, execute_batch

# execute_values (INSERT massif, très rapide)
execute_values(cur,
    "INSERT INTO t (a, b) VALUES %s",
    [(1,'x'), (2,'y'), (3,'z')],
    page_size=1000)              Taille des batches

execute_values(cur,
    "INSERT INTO t (a, b) VALUES %s RETURNING id",
    données,
    fetch=True)                  Récupérer les ids insérés

# execute_batch (executeMany optimisé)
execute_batch(cur,
    "UPDATE t SET b=%s WHERE a=%s",
    [(v, k) for k, v in données],
    page_size=500)

# copy_from (COPY ultra-rapide)
import io
données_io = io.StringIO("1\talice\n2\tbob\n")
cur.copy_from(données_io, 'utilisateurs', columns=('id', 'nom'))

# copy_to
sortie = io.StringIO()
cur.copy_to(sortie, 'utilisateurs')
contenu = sortie.getvalue()

# copy_expert (COPY avec format custom)
cur.copy_expert("COPY t FROM STDIN WITH (FORMAT csv, HEADER)", fichier_csv)
cur.copy_expert("COPY t TO STDOUT WITH (FORMAT csv)", sortie)

— EXTRAS : TYPES SPÉCIAUX —
from psycpg2.extras import (
    Json, RealDictCursor, DictCursor, NamedTupleCursor,
    Inet, DateTimeTZRange, DateRange, NumericRange, execute_values
)

# JSON
cur.execute("INSERT INTO t (data) VALUES %s", (Json({'a': 1}),))
cur.execute("SELECT data FROM t WHERE data->>'clé' = %s", ('val',))

# UUID
import uuid
psycpg2.extras.register_uuid()
cur.execute("INSERT INTO t (id) VALUES %s", (uuid.uuid4(),))

# hstore
psycpg2.extras.register_hstore(conn)
cur.execute("INSERT INTO t (h) VALUES %s", ({'a': '1', 'b': '2'},))

```

```

# Types composites
psycpg2.extras.register_composite('mon_type', conn)

# Tableau PostgreSQL
cur.execute("SELECT %s::integer[]", ([1, 2, 3],))
cur.execute("INSERT INTO t (arr) VALUES %s", ([1, 2, 3],))

# Types géographiques (PostGIS via psycpg2-binary)
from psycpg2.extras import Inet
cur.execute("INSERT INTO t (ip) VALUES %s", (Inet('192.168.1.1'),))

# Plages (Range types)
from psycpg2.extras import NumericRange, DateRange, DateTimeTZRange
cur.execute("INSERT INTO t (r) VALUES %s", (NumericRange(1, 10),))

# Notification LISTEN/NOTIFY
conn.set_isolation_level(0)                Autocommit requis
cur.execute("LISTEN mon_canal")
import select
select.select([conn], [], [], 5)            Attendre 5s
conn.poll()
for notif in conn.notifies:
    print(notif.channel, notif.payload)
conn.notifies.clear()
# Envoyer une notification
cur.execute("NOTIFY mon_canal, 'message'")

```

— POOL DE CONNEXIONS —

```

from psycpg2 import pool

# Pool simple (non thread-safe)
pool_simple = pool.SimpleConnectionPool(
    minconn=1, maxconn=10,
    host="localhost", database="mabase",
    user="user", password="pass"
)
conn = pool_simple.getconn()
pool_simple.putconn(conn)
pool_simple.closeall()

# Pool thread-safe
pool_thread = pool.ThreadedConnectionPool(
    minconn=2, maxconn=20,
    host="localhost", database="mabase",
    user="user", password="pass"
)
conn = pool_thread.getconn()
pool_thread.putconn(conn)                Retourner au pool
pool_thread.putconn(conn, close=True)     Retourner et fermer
pool_thread.closeall()

# Pool persistant (maintenu entre requêtes)
pool_persist = pool.PersistentConnectionPool(
    minconn=1, maxconn=10, ...
)

```

— ADAPTATION DE TYPES —

```

from psycpg2.extensions import (
    adapt, register_adapter, new_type, register_type, AsIs, QuotedString
)

# Adapter un type custom
class MonType:
    def __init__(self, val):
        self.val = val

def adapter_mon_type(obj):
    return AsIs(f"'{obj.val}'")

register_adapter(MonType, adapter_mon_type)

# Enregistrer un type entrant
OID_MON_TYPE = 12345
mon_type_pg = new_type((OID_MON_TYPE,), "MON_TYPE",

```

```

        lambda val, cur: MonType(val))
register_type(mon_type_pg)

# Convertir en texte SQL
adapt(42)                Integer(42)
adapt('texte')           QuotedString
adapt(None)              ISQLQuote(None)

```

— COMPOSITION DE REQUÊTES —

```

from psycopg2 import sql

# Identifiants (table, colonne) – ne pas utiliser %s pour les noms
query = sql.SQL("SELECT {} FROM {}".format(
    sql.Identifier('colonne'),
    sql.Identifier('ma_table')
))
query = sql.SQL("SELECT {col} FROM {table}").format(
    col=sql.Identifier('nom'),
    table=sql.Identifier('utilisateurs')
)

# Liste de colonnes
query = sql.SQL("SELECT {} FROM t").format(
    sql.SQL(', ').join(map(sql.Identifier, ['a', 'b', 'c']))
)

# Littéraux
query = sql.SQL("SELECT {} FROM t WHERE id = {}".format(
    sql.Identifier('nom'),
    sql.Literal(42)
))

# Schéma + table
sql.Identifier('schéma', 'table')          "schéma"."table"

cur.execute(query)
cur.execute(query, params)

```

— ERREURS ET EXCEPTIONS —

psycopg2.Error	Exception de base
psycopg2.DatabaseError	Erreur base de données
psycopg2.OperationalError	Connexion perdue, timeout...
psycopg2.ProgrammingError	Erreur SQL syntaxe / objet inexistant
psycopg2.IntegrityError	Violation contrainte (FK, UNIQUE...)
psycopg2.DataError	Valeur hors plage, type invalide
psycopg2.NotSupportedError	Fonctionnalité non supportée
psycopg2.InternalError	Erreur interne PostgreSQL
psycopg2.InterfaceError	Erreur interface Python
psycopg2.Warning	Avertissement DB-API

```

try:
    cur.execute("INSERT INTO t VALUES (%s)", (val,))
    conn.commit()
except psycopg2.IntegrityError as e:
    conn.rollback()
    print(e.pgcode)           Code d'erreur PostgreSQL (5 chiffres)
    print(e.pgerror)         Message d'erreur complet
    print(e.diag.message_primary) Message principal
    print(e.diag.detail)     Détails
    print(e.diag.hint)       Conseil
    print(e.diag.column_name) Colonne impliquée
    print(e.diag.constraint_name) Nom de la contrainte
    print(e.diag.table_name)  Nom de la table
except psycopg2.OperationalError as e:
    # Connexion perdue – reconnecter
    pass

# Codes d'erreur courants
# 23505 : unique_violation (UNIQUE)
# 23503 : foreign_key_violation (FK)
# 23502 : not_null_violation
# 23514 : check_violation
# 42P01 : undefined_table
# 42703 : undefined_column
# 08006 : connection_failure

```

```
# 40001 : serialization_failure
# 40P01 : deadlock_detected
```

— EXEMPLES COMPLETS —

```
import psycopg2
from psycopg2.extras import RealDictCursor, execute_values
from contextlib import contextmanager

DSN = "postgresql://user:pass@localhost/mabase"

@contextmanager
def get_cursor(factory=None):
    conn = psycopg2.connect(DSN)
    try:
        kw = {'cursor_factory': factory} if factory else {}
        cur = conn.cursor(**kw)
        yield cur
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

# SELECT
with get_cursor(RealDictCursor) as cur:
    cur.execute("SELECT * FROM utilisateurs WHERE actif = %s", (True,))
    utilisateurs = cur.fetchall()

# INSERT massif
données = [(f"user{i}", i*10) for i in range(10000)]
with get_cursor() as cur:
    execute_values(cur,
        "INSERT INTO utilisateurs (nom, score) VALUES %s",
        données, page_size=500)

# INSERT avec RETURNING
with get_cursor() as cur:
    cur.execute(
        "INSERT INTO articles (titre, contenu) VALUES (%s, %s) RETURNING id, created_at",
        ("Mon titre", "Contenu...")
    )
    résultat = cur.fetchone()
    nouvel_id = résultat[0]

# Transaction explicite
conn = psycopg2.connect(DSN)
try:
    with conn:
        cur = conn.cursor()
        cur.execute("UPDATE comptes SET solde = solde - %s WHERE id = %s", (100, 1))
        cur.execute("UPDATE comptes SET solde = solde + %s WHERE id = %s", (100, 2))
finally:
    conn.close()
```

[pg] psycopg2 – Types de données PostgreSQL

DESCRIPTION : Correspondances types Python ↔ PostgreSQL

Python → PostgreSQL :

None	NULL
bool	BOOLEAN
int	INTEGER / BIGINT / SMALLINT
float	REAL / DOUBLE PRECISION
Decimal	NUMERIC
str	TEXT / VARCHAR / CHAR
bytes / bytearray	BYTEA
datetime.date	DATE
datetime.time	TIME / TIMETZ
datetime.datetime	TIMESTAMP / TIMESTAMPTZ
datetime.timedelta	INTERVAL
list / tuple	ARRAY (si homogène)
dict (Json())	JSON / JSONB

uuid.UUID	UUID
Inet	INET / CIDR
Range types	int4range, numrange, daterange...
memoryview	BYTEA

PostgreSQL → Python :

NULL	None
BOOLEAN	bool
SMALLINT/INT/BIGINT	int
REAL/DOUBLE	float
NUMERIC/DECIMAL	Decimal (si register_default_jsonb)
TEXT/VARCHAR/CHAR	str
BYTEA	memoryview
DATE	datetime.date
TIME	datetime.time
TIMESTAMP	datetime.datetime (naïf)
TIMESTAMPZ	datetime.datetime (avec tzinfo)
INTERVAL	datetime.timedelta
ARRAY	list
JSON/JSONB	dict / list / str / int / float / bool / None
UUID	str (par défaut) ou uuid.UUID (après register_uuid)
OID	int

[my] PyMySQL – Connexion MySQL / MariaDB (pur Python)

DESCRIPTION : Client MySQL pur Python, compatible DB-API 2.0
 POURQUOI : Pas de dépendance C, compatible AWS RDS, Azure, Google Cloud SQL
 CONTEXTE : Applications web, microservices, cloud

— CONNEXION —

```
import pymysql
import pymysql.cursors

conn = pymysql.connect(
    host='localhost',
    port=3306,
    database='mabase',
    user='utilisateur',
    password='motdepasse',
)

conn = pymysql.connect(
    host='localhost',
    port=3306,
    db='mabase',
    user='user',
    passwd='pass',
    charset='utf8mb4',
    collation='utf8mb4_unicode_ci',
    connect_timeout=10,
    read_timeout=30,
    write_timeout=30,
    init_command='SET time_zone="+00:00"',
    autocommit=False,
    cursorclass=pymysql.cursors.DictCursor,
    ssl={'ca': '/chemin/ca.pem',
        'cert': '/chemin/client.crt',
        'key': '/chemin/client.key'},
    ssl_verify_cert=True,
    ssl_verify_identity=True,
    local_infile=False,
    program_name='mon_app',
    defer_connect=False,
    max_allowed_packet=16*1024*1024,
    sql_mode='STRICT_TRANS_TABLES',
    client_flag=0,
    conv={},
    read_default_file='~/my.cnf',
    read_default_group='client',
    bind_address=None,
    unix_socket=None,
    compress=False,
)
```

Alias de database

Alias de password

Encodage (recommandé)

Timeout lecture (secondes)

Timeout écriture

Commande d'initialisation

Mode autocommit

Curseur par défaut

SSL

LOAD DATA LOCAL INFILE

Connexion différée

Taille max paquet (16 Mo)

Conversions de types custom

Fichier de conf MySQL

Groupe dans le fichier de conf

Adresse locale

Socket Unix (localhost)

Compresser les données

```

conn.close()
conn.ping(reconnect=True)                Vérifier / reconnecter si nécessaire

```

— TRANSACTION —

```

conn.commit()
conn.rollback()
conn.autocommit(True)                    Activer autocommit
conn.autocommit(False)                   Désactiver
conn.get_autocommit()                    Lire l'état autocommit
conn.begin()                             Démarrer transaction explicite
# Savepoints
cur.execute("SAVEPOINT sp1")
cur.execute("ROLLBACK TO SAVEPOINT sp1")
cur.execute("RELEASE SAVEPOINT sp1")

```

— CURSEURS —

```

cur = conn.cursor()                      Curseur standard (tuples)
cur = conn.cursor(pymysql.cursors.DictCursor)    dict
cur = conn.cursor(pymysql.cursors.SSCursor)      Streaming (unbuffered)
cur = conn.cursor(pymysql.cursors.SSDictCursor)  Streaming dict

cur.close()
cur.connection                           Connexion parente
cur.rowcount                             Nb lignes affectées
cur.lastrowid                            ID auto-increment dernière insertion
cur.description                           Métadonnées colonnes
cur.rownumber                             Numéro de ligne courant

```

— EXÉCUTION —

```

cur.execute(sql)
cur.execute("SELECT %s, %s", (1, 'texte'))      Paramètres positionnels %s
cur.execute("SELECT %(nom)s", {'nom': 'val'})    Paramètres nommés
cur.executemany(sql, liste_params)
cur.executemany("INSERT INTO t VALUES (%s,%s)", [(1,'a'),(2,'b')])
cur.callproc('ma_procedure', (arg1, arg2))
cur.mogrify("SELECT %s", (42,))                 Voir la requête rendue (debug)

# IMPORTANT : Utiliser %s pour les paramètres (pas de f-string)
# x f"INSERT INTO t VALUES ({val})"             Injection SQL !
# ✓ "INSERT INTO t VALUES (%s)", (val,)

```

— FETCH —

```

cur.fetchone()                             Une ligne ou None
cur.fetchmany(size=10)                     N lignes
cur.fetchall()                             Toutes les lignes
for ligne in cur:                           Itération directe

```

— SSCURSOR (streaming, gros résultats) —

```

cur = conn.cursor(pymysql.cursors.SSCursor)
cur.execute("SELECT * FROM grande_table")
while True:
    ligne = cur.fetchone()
    if ligne is None:
        break
    traiter(ligne)

```

— INFORMATIONS DE CONNEXION —

```

conn.get_server_info()                     Version serveur (str)
conn.server_version                        Version (int ex: 80031)
conn.thread_id()                           ID du thread connexion
conn.character_set_name()                  Jeu de caractères
conn.db                                    Nom de la base
conn.host                                  Hôte
conn.port                                  Port
conn.user                                  Utilisateur
conn.open                                  True si connecté
conn.affected_rows()                       Lignes affectées dernière requête
conn.insert_id()                           Dernier auto-increment ID
conn.warning_count()                       Nombre d'avertissements
cur.execute("SHOW WARNINGS")                Détails des avertissements

```

— EXEMPLES COMPLETS —

```

import pymysql
import pymysql.cursors
from contextlib import contextmanager

```



```

CONFIG = dict(host='localhost', database='mabase',
              user='user', password='pass', charset='utf8mb4',
              cursorclass=pymysql.cursors.DictCursor)

@contextmanager
def get_conn():
    conn = pymysql.connect(**CONFIG)
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

# SELECT
with get_conn() as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM produits WHERE prix > %s", (10.0,))
        produits = cur.fetchall()

# INSERT avec id retourné
with get_conn() as conn:
    with conn.cursor() as cur:
        cur.execute(
            "INSERT INTO utilisateurs (nom, email) VALUES (%s, %s)",
            ("Alice", "alice@example.com")
        )
        nouvel_id = cur.lastrowid

# INSERT massif
données = [(f"user{i}", i) for i in range(1000)]
with get_conn() as conn:
    with conn.cursor() as cur:
        cur.executemany(
            "INSERT INTO t (nom, score) VALUES (%s, %s)", données)

```

[my] mysql-connector-python – Client officiel MySQL (Oracle)
--

DESCRIPTION : Client officiel MySQL par Oracle, C-extension ou pur Python
 POURQUOI : Support officiel, X DevAPI, compression, meilleure intégration SSL
 INSTALL : pip install mysql-connector-python

— CONNEXION

```

import mysql.connector

conn = mysql.connector.connect(
    host='localhost',
    port=3306,
    database='mabase',
    user='user',
    password='pass',
    charset='utf8mb4',
    collation='utf8mb4_unicode_ci',
    autocommit=False,
    connection_timeout=10,
    use_unicode=True,
    use_pure=False,
    ssl_ca='/chemin/ca.pem',
    ssl_cert='/chemin/client.crt',
    ssl_key='/chemin/client.key',
    ssl_verify_cert=True,
    ssl_verify_identity=True,
    tls_versions=['TLSv1.3'],
    get_warnings=True,
    raise_on_warnings=True,
    consume_results=False,
    allow_local_infile=False,
    client_flags=[mysql.connector.ClientFlag.SSL],
    time_zone='+00:00',
    sql_mode='STRICT_TRANS_TABLES',

```

False = C-extension (plus rapide)

Récupérer les warnings
Lever exception sur warning

```

    auth_plugin='mysql_native_password',
    kerberos_auth_mode=None,
    pool_name='mypool',                Pool de connexions
    pool_size=5,
    pool_reset_session=True,
)

# Depuis un dictionnaire de config
config = {'host': 'localhost', 'database': 'mabase', ...}
conn = mysql.connector.connect(**config)

conn.close()
conn.is_connected()                    True si connecté
conn.reconnect(attempts=3, delay=2)    Reconnecter

```

CURSEURS

```

cur = conn.cursor()                    Tuples
cur = conn.cursor(dictionary=True)     Dicts
cur = conn.cursor(named_tuple=True)    NamedTuples
cur = conn.cursor(buffered=True)       Résultats en mémoire (défaut)
cur = conn.cursor(buffered=False)      Streaming (unbuffered)
cur = conn.cursor(raw=True)            Types bruts (bytes)
cur = conn.cursor(prepared=True)       Requêtes préparées côté serveur

```

EXÉCUTION

```

cur.execute(sql)
cur.execute("SELECT %s, %s", (1, 'texte'))
cur.execute("SELECT %(nom)s", {'nom': 'val'})
cur.executemany(sql, liste_params)
cur.callproc('ma_procedure', (arg1, arg2))
cur.stored_results()                  Résultats des procédures stockées

```

REQUÊTES PRÉPARÉES

```

cur = conn.cursor(prepared=True)
cur.execute("SELECT * FROM t WHERE id = %s AND actif = %s", (42, True))
cur.execute("INSERT INTO t VALUES (%s, %s)", (1, 'texte'))
# La requête est préparée côté serveur à la première exécution

```

POOL DE CONNEXIONS

```

from mysql.connector import pooling

pool = pooling.MySQLConnectionPool(
    pool_name='monpool',
    pool_size=5,
    pool_reset_session=True,
    host='localhost',
    database='mabase',
    user='user',
    password='pass',
)
conn = pool.get_connection()
conn.close()                          Retourne au pool automatiquement

```

INFORMATIONS

```

conn.get_server_info()                 Version serveur
conn.server_version                    (major, minor, patch)
conn.connection_id                     ID de connexion
conn.database                          Base courante
cur.lastrowid                          Dernier auto-increment
cur.rowcount                           Lignes affectées
cur.description                       Métadonnées colonnes
conn.get_warnings()                   Liste des warnings
cur.fetchwarnings()                    Warnings du curseur

```

EXCEPTIONS

```

mysql.connector.Error                  Base
mysql.connector.InterfaceError
mysql.connector.DatabaseError
mysql.connector.DataError
mysql.connector.OperationalError
mysql.connector.IntegrityError         Contrainte FK, UNIQUE...
mysql.connector.InternalError
mysql.connector.ProgrammingError
mysql.connector.NotSupportedError

```

```

try:
    cur.execute(sql, params)
except mysql.connector.IntegrityError as e:
    print(e.errno)           Code erreur MySQL
    print(e.sqlstate)       SQL State (5 chars)
    print(e.msg)            Message
    # 1062 : Duplicate entry (UNIQUE)
    # 1452 : FK constraint failed
    # 1048 : Column cannot be null
    # 1054 : Unknown column

```

[sql] SQL commun – DDL, DML, requêtes, optimisation

DESCRIPTION : SQL standard applicable à PostgreSQL et MySQL/MariaDB
POURQUOI : Référence rapide des opérations SQL courantes
CONTEXTE : Toutes les opérations CRUD, schéma, index

— DDL (Data Definition Language) —

```

-- Créer une table
CREATE TABLE utilisateurs (
    id          SERIAL PRIMARY KEY,          -- PostgreSQL auto-incrément
    -- id       INT AUTO_INCREMENT PRIMARY KEY, -- MySQL
    nom         VARCHAR(100) NOT NULL,
    email       VARCHAR(255) UNIQUE NOT NULL,
    age         INTEGER CHECK (age >= 0),
    actif       BOOLEAN DEFAULT TRUE,
    créé_le     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    modifié_le  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS ma_table (...);
DROP TABLE ma_table;
DROP TABLE IF EXISTS ma_table;
DROP TABLE IF EXISTS ma_table CASCADE;      -- PostgreSQL
TRUNCATE TABLE ma_table;
TRUNCATE TABLE ma_table RESTART IDENTITY;    -- PostgreSQL

ALTER TABLE t ADD COLUMN col VARCHAR(50);
ALTER TABLE t ADD COLUMN col VARCHAR(50) NOT NULL DEFAULT '';
ALTER TABLE t DROP COLUMN col;
ALTER TABLE t DROP COLUMN IF EXISTS col;
ALTER TABLE t RENAME COLUMN ancien TO nouveau;
ALTER TABLE t ALTER COLUMN col TYPE TEXT;    -- PostgreSQL
ALTER TABLE t MODIFY COLUMN col TEXT;        -- MySQL
ALTER TABLE t ALTER COLUMN col SET DEFAULT 0; -- PostgreSQL
ALTER TABLE t ALTER COLUMN col DROP DEFAULT; -- PostgreSQL
ALTER TABLE t ALTER COLUMN col SET NOT NULL;  -- PostgreSQL
ALTER TABLE t ALTER COLUMN col DROP NOT NULL; -- PostgreSQL
ALTER TABLE t RENAME TO nouveau_nom;
ALTER TABLE t ADD CONSTRAINT fk_t FOREIGN KEY (col_id) REFERENCES autre(id)
    ON DELETE CASCADE ON UPDATE CASCADE;
ALTER TABLE t ADD CONSTRAINT chk CHECK (col > 0);
ALTER TABLE t ADD CONSTRAINT uq UNIQUE (col1, col2);
ALTER TABLE t DROP CONSTRAINT nom_contrainte; -- PostgreSQL
ALTER TABLE t DROP FOREIGN KEY nom_contrainte; -- MySQL

-- Index
CREATE INDEX idx_nom ON t(col);
CREATE UNIQUE INDEX idx_nom ON t(col);
CREATE INDEX idx_nom ON t(col1, col2);
CREATE INDEX idx_nom ON t(col) WHERE actif = TRUE; -- PostgreSQL (partiel)
CREATE INDEX CONCURRENTLY idx_nom ON t(col);        -- PostgreSQL (sans lock)
DROP INDEX idx_nom;                                 -- PostgreSQL
DROP INDEX IF EXISTS idx_nom;
DROP INDEX t.idx_nom;                               -- MySQL
REINDEX INDEX idx_nom;                              -- PostgreSQL

```

— DML (Data Manipulation Language) —

```

-- INSERT
INSERT INTO t (a, b) VALUES (1, 'texte');
INSERT INTO t (a, b) VALUES (1, 'x'), (2, 'y'), (3, 'z'); Batch insert
INSERT INTO t DEFAULT VALUES;                  Toutes valeurs par défaut

```

```

-- PostgreSQL : RETURNING
INSERT INTO t (a, b) VALUES (%s, %s) RETURNING id, créé_le;

-- PostgreSQL : ON CONFLICT (UPSERT)
INSERT INTO t (id, val) VALUES (%s, %s)
ON CONFLICT (id) DO NOTHING;
INSERT INTO t (id, val) VALUES (%s, %s)
ON CONFLICT (id) DO UPDATE SET val = EXCLUDED.val;
INSERT INTO t (id, val) VALUES (%s, %s)
ON CONFLICT (id) DO UPDATE SET val = EXCLUDED.val, modifié_le = NOW()
WHERE t.val IS DISTINCT FROM EXCLUDED.val;

-- MySQL : ON DUPLICATE KEY (UPSERT)
INSERT INTO t (id, val) VALUES (%s, %s)
ON DUPLICATE KEY UPDATE val = VALUES(val);
INSERT INTO t (id, val) VALUES (%s, %s)
ON DUPLICATE KEY UPDATE val = val + 1;

-- REPLACE INTO (MySQL : delete + insert)
REPLACE INTO t (id, val) VALUES (%s, %s);

-- UPDATE
UPDATE t SET col = %s WHERE id = %s;
UPDATE t SET a = %s, b = %s WHERE id = %s;
UPDATE t SET col = col + 1 WHERE actif = TRUE;
UPDATE t SET col = DEFAULT WHERE id = %s;
UPDATE t1 SET t1.col = t2.col FROM t2 WHERE t1.id = t2.id; PostgreSQL
UPDATE t1, t2 SET t1.col = t2.col WHERE t1.id = t2.id; MySQL
UPDATE t SET col = %s RETURNING *; PostgreSQL

-- DELETE
DELETE FROM t WHERE id = %s;
DELETE FROM t WHERE id IN (%s, %s, %s);
DELETE FROM t USING t2 WHERE t.id = t2.id; PostgreSQL
DELETE t1 FROM t1 JOIN t2 ON t1.id = t2.id; MySQL
DELETE FROM t RETURNING id; PostgreSQL

-- SELECT
SELECT * FROM t;
SELECT a, b, c FROM t;
SELECT DISTINCT a FROM t;
SELECT t.a, t.b FROM t;
SELECT a AS alias FROM t;
SELECT a, COUNT(*) AS nb FROM t GROUP BY a;

-- WHERE
WHERE col = %s
WHERE col != %s
WHERE col > %s AND col < %s
WHERE col BETWEEN %s AND %s
WHERE col IN (%s, %s, %s)
WHERE col NOT IN (SELECT ...)
WHERE col IS NULL
WHERE col IS NOT NULL
WHERE col LIKE %s 'motif%' ou '%motif%'
WHERE col ILIKE %s Insensible casse (PostgreSQL)
WHERE col ~* %s Regex insensible casse (PostgreSQL)
WHERE col ~ %s Regex sensible casse (PostgreSQL)
WHERE col REGEXP %s Regex (MySQL)
WHERE col SIMILAR TO %s SIMILAR TO (PostgreSQL)

-- JOIN
INNER JOIN t2 ON t1.id = t2.t1_id
LEFT JOIN t2 ON t1.id = t2.t1_id
RIGHT JOIN t2 ON t1.id = t2.t1_id
FULL OUTER JOIN t2 ON t1.id = t2.t1_id PostgreSQL
CROSS JOIN t2
NATURAL JOIN t2
JOIN t2 USING (colonne_commune)

-- ORDER, LIMIT, OFFSET
ORDER BY col ASC
ORDER BY col DESC
ORDER BY col DESC NULLS LAST PostgreSQL
LIMIT 10

```

LIMIT 10 OFFSET 20
LIMIT %s OFFSET %s

-- GROUP BY / HAVING
GROUP BY col1, col2
HAVING COUNT(*) > 5
GROUP BY ROLLUP (col1, col2) PostgreSQL
GROUP BY CUBE (col1, col2) PostgreSQL
GROUP BY GROUPING SETS ((col1), (col2), ()) PostgreSQL

-- Fonctions d'agrégation
COUNT(*), COUNT(col), COUNT(DISTINCT col)
SUM(col), AVG(col), MIN(col), MAX(col)
STRING_AGG(col, ', ') PostgreSQL
GROUP_CONCAT(col SEPARATOR ', ') MySQL
ARRAY_AGG(col) PostgreSQL
JSON_AGG(col) PostgreSQL
BOOL_AND(col), BOOL_OR(col) PostgreSQL
STDDEV(col), VARIANCE(col)

-- Sous-requêtes
SELECT * FROM t WHERE id IN (SELECT id FROM t2 WHERE ...)
SELECT * FROM t WHERE EXISTS (SELECT 1 FROM t2 WHERE t2.t_id = t.id)
SELECT * FROM (SELECT ...) AS sous_req
WITH cte AS (SELECT ...) SELECT * FROM cte CTE (Common Table Expression)
WITH RECURSIVE cte AS (
 SELECT id, parent_id, 1 AS niveau FROM t WHERE parent_id IS NULL
 UNION ALL
 SELECT t.id, t.parent_id, cte.niveau + 1
 FROM t JOIN cte ON t.parent_id = cte.id
) SELECT * FROM cte;

-- Fenêtrage (Window Functions)
ROW_NUMBER() OVER (PARTITION BY col ORDER BY autre)
RANK() OVER (PARTITION BY col ORDER BY autre)
DENSE_RANK() OVER (...)
NTILE(4) OVER (ORDER BY col)
LAG(col, 1) OVER (ORDER BY autre)
LEAD(col, 1) OVER (ORDER BY autre)
FIRST_VALUE(col) OVER (...)
LAST_VALUE(col) OVER (...)
SUM(col) OVER (PARTITION BY grp ORDER BY dt ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

-- CASE
CASE WHEN col > 0 THEN 'positif' WHEN col < 0 THEN 'négatif' ELSE 'zéro' END
CASE col WHEN 1 THEN 'un' WHEN 2 THEN 'deux' ELSE 'autre' END

-- COALESCE / NULLIF / GREATEST / LEAST
COALESCE(col, 0) Premier non-NULL
NULLIF(col, 0) NULL si égal à 0
GREATEST(a, b, c) Maximum
LEAST(a, b, c) Minimum

— FONCTIONS UTILES —

-- Dates/temps PostgreSQL
NOW() Timestamp courant avec TZ
CURRENT_TIMESTAMP Idem
CURRENT_DATE Date courante
CURRENT_TIME Heure courante
date_trunc('month', ts) Tronquer au mois
date_trunc('day', ts) Tronquer au jour
EXTRACT(EPOCH FROM ts) Timestamp UNIX
EXTRACT(YEAR FROM ts) Année
ts + INTERVAL '1 day' Ajouter 1 jour
ts + INTERVAL '2 hours' Ajouter 2 heures
AGE(ts1, ts2) Différence comme INTERVAL
TO_CHAR(ts, 'YYYY-MM-DD HH24:MI:SS') Formater
TO_TIMESTAMP(epoch) UNIX → timestamp
-- Dates/temps MySQL
NOW() Datetime courante
CURDATE() Date courante
CURTIME() Heure courante
DATE_FORMAT(ts, '%Y-%m-%d') Formater
DATE_ADD(dt, INTERVAL 1 DAY) Ajouter
DATEDIFF(dt1, dt2) Différence en jours

UNIX_TIMESTAMP(dt)	→ UNIX timestamp
FROM_UNIXTIME(ts)	UNIX → datetime
-- Texte commun	
UPPER(col), LOWER(col), LENGTH(col)	
TRIM(col), LTRIM(col), RTRIM(col)	
SUBSTRING(col, 1, 5)	
CONCAT(a, b, c), CONCAT_WS('-', a, b)	
REPLACE(col, 'ancien', 'nouveau')	
POSITION('sub' IN col)	
SPLIT_PART(col, '.', 1)	PostgreSQL
REGEXP_REPLACE(col, 'pat', 'rem')	PostgreSQL
FORMAT('%s monde', 'bonjour')	PostgreSQL
-- Numériques	
ROUND(col, 2), FLOOR(col), CEIL(col)	
ABS(col), MOD(col, 5)	
POWER(col, 2), SQRT(col)	
RANDOM()	PostgreSQL (0.0–1.0)
RAND()	MySQL
GEN_RANDOM_UUID()	PostgreSQL (UUID v4)
UUID()	MySQL
-- JSON PostgreSQL	
col->'clé'	Valeur JSON (JSON)
col->>'clé'	Valeur JSON (texte)
col#>'{a,b}'	Chemin JSON
col#>>'{a,b}'	Chemin JSON (texte)
col @> '{"a":1}'	Contient
col <@ '{"a":1}'	Contenu dans
col ? 'clé'	Clé existe
jsonb_set(col, '{clé}', '"val"')	Modifier
jsonb_array_elements(col)	Développer tableau JSON
json_build_object('k', v)	Construire objet JSON
json_agg(ligne)	Agréger en JSON
-- JSON MySQL	
JSON_EXTRACT(col, '\$.clé')	
JSON_VALUE(col, '\$.clé')	
JSON_CONTAINS(col, '"val"', '\$.clé')	
JSON_SET(col, '\$.clé', 'val')	
JSON_ARRAY(a, b, c)	
JSON_OBJECT('k', v)	

[py] Patterns communs – Context managers, pooling, migrations

```

— CONTEXT MANAGER UNIVERSEL —
from contextlib import contextmanager
import psycopg2, pymysql

# Générique – fonctionne avec psycopg2 et pymysql
@contextmanager
def transaction(conn):
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise

with transaction(conn) as c:
    cur = c.cursor()
    cur.execute(...)

```

```

— RETRY SUR CONNEXION PERDUE —
import time, psycopg2

def execute_avec_retry(conn_factory, sql, params=(), retries=3, délai=1):
    for tentative in range(retries):
        try:
            conn = conn_factory()
            cur = conn.cursor()

```

```

        cur.execute(sql, params)
        conn.commit()
        return cur.fetchall()
    except psycopg2.OperationalError:
        if tentative < retries - 1:
            time.sleep(délai * (2 ** tentative))
        else:
            raise
    finally:
        try:
            conn.close()
        except Exception:
            pass

```

PAGINATION

```

def paginer(cur, sql, params, page=1, taille=20):
    offset = (page - 1) * taille
    cur.execute(sql + " LIMIT %s OFFSET %s", (*params, taille, offset))
    return cur.fetchall()

# PostgreSQL keyset pagination (plus performant)
cur.execute(
    "SELECT * FROM t WHERE id > %s ORDER BY id LIMIT %s",
    (dernier_id, taille)
)

```

BULK INSERT OPTIMISÉ

```

# PostgreSQL : execute_values (le plus rapide)
from psycopg2.extras import execute_values
execute_values(cur, "INSERT INTO t (a,b) VALUES %s", données, page_size=1000)

# PostgreSQL : COPY (encore plus rapide)
import io, csv
buf = io.StringIO()
writer = csv.writer(buf)
writer.writerows(données)
buf.seek(0)
cur.copy_from(buf, 't', columns=('a', 'b'), sep=',')

# MySQL : executemany
cur.executemany("INSERT INTO t (a,b) VALUES (%s,%s)", données)

# MySQL : INSERT multi-lignes
placeholders = ','.join(['(%s,%s)'] * len(données))
valeurs = [v for row in données for v in row]
cur.execute(f"INSERT INTO t (a,b) VALUES {placeholders}", valeurs)

```

MIGRATIONS SIMPLES

```

def appliquer_migration(conn, version, sql):
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS migrations (
            version INTEGER PRIMARY KEY,
            appliqué_le TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)
    cur.execute("SELECT version FROM migrations WHERE version = %s", (version,))
    if cur.fetchone():
        return False
    cur.execute(sql)
    cur.execute("INSERT INTO migrations (version) VALUES (%s)", (version,))
    conn.commit()
    return True

```

INTROSPECTION SCHÉMA

```

-- PostgreSQL : lister les tables
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE';

-- PostgreSQL : colonnes d'une table
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_name = 'ma_table' ORDER BY ordinal_position;

-- PostgreSQL : index

```

```

SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'ma_table';

-- PostgreSQL : taille des tables
SELECT relname, pg_size_pretty(pg_total_relation_size(relid))
FROM pg_catalog.pg_statio_user_tables ORDER BY pg_total_relation_size(relid) DESC;

-- MySQL : lister les tables
SHOW TABLES;
SELECT table_name FROM information_schema.tables
WHERE table_schema = DATABASE();

-- MySQL : colonnes d'une table
DESCRIBE ma_table;
SHOW COLUMNS FROM ma_table;

-- MySQL : index
SHOW INDEX FROM ma_table;

-- MySQL : taille des tables
SELECT table_name, ROUND((data_length + index_length) / 1024 / 1024, 2) AS 'MB'
FROM information_schema.tables WHERE table_schema = DATABASE()
ORDER BY (data_length + index_length) DESC;

```

```

-- EXPLAIN / ANALYZE
-- PostgreSQL
EXPLAIN SELECT * FROM t WHERE col = 'val';
EXPLAIN ANALYZE SELECT * FROM t WHERE col = 'val';
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) SELECT ...;

-- MySQL
EXPLAIN SELECT * FROM t WHERE col = 'val';
EXPLAIN FORMAT=JSON SELECT ...;
EXPLAIN ANALYZE SELECT ...;          -- MySQL 8.0+

```

```

-- SURVEILLANCE
-- PostgreSQL : requêtes en cours
SELECT pid, now() - pg_stat_activity.query_start AS durée,
       query, state
FROM pg_stat_activity
WHERE state != 'idle' AND query_start < now() - interval '5 minutes';

-- PostgreSQL : verrous
SELECT * FROM pg_locks WHERE NOT granted;

-- PostgreSQL : statistiques tables
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables;

-- MySQL : requêtes en cours
SHOW PROCESSLIST;
SELECT * FROM information_schema.PROCESSLIST WHERE TIME > 5;

-- MySQL : statut
SHOW STATUS LIKE 'Connections';
SHOW GLOBAL STATUS LIKE 'Slow_queries';
SHOW VARIABLES LIKE 'max_connections';

```

[py] SQLAlchemy Core & ORM – Aperçu (surcouche)

```

DESCRIPTION : Surcouche ORM/Core compatible PostgreSQL et MySQL
POURQUOI    : Abstraction du dialecte, pool intégré, ORM, migrations (Alembic)
INSTALL     : pip install sqlalchemy psycopg2-binary pymysql

```

```

-- CONNEXION
from sqlalchemy import create_engine, text

# PostgreSQL
engine = create_engine("postgresql+psycopg2://user:pass@localhost/mabase")
engine = create_engine("postgresql+psycopg2://user:pass@localhost/mabase",
                       pool_size=5, max_overflow=10, pool_timeout=30, pool_recycle=3600)

# MySQL
engine = create_engine("mysql+pymysql://user:pass@localhost/mabase?charset=utf8mb4")

```



```

engine = create_engine("mysql+mysqlconnector://user:pass@localhost/mabase")

# Utilisation
with engine.connect() as conn:
    résultat = conn.execute(text("SELECT * FROM t WHERE id = :id"), {'id': 42})
    for ligne in résultat:
        print(ligne)

with engine.begin() as conn:
    conn.execute(text("INSERT INTO t VALUES (:a, :b)"), {'a': 1, 'b': 'x'})

```

— ORM BASIQUE —

```

from sqlalchemy.orm import DeclarativeBase, Session, mapped_column, Mapped
from sqlalchemy import String, Integer, select

class Base(DeclarativeBase):
    pass

class Utilisateur(Base):
    __tablename__ = 'utilisateurs'
    id: Mapped[int] = mapped_column(primary_key=True)
    nom: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(255), unique=True)

Base.metadata.create_all(engine)

with Session(engine) as session:
    # Ajouter
    user = Utilisateur(nom='Alice', email='alice@ex.com')
    session.add(user)
    session.commit()

    # Requête
    users = session.execute(
        select(Utilisateur).where(Utilisateur.nom == 'Alice')
    ).scalars().all()

    # Modifier
    user.nom = 'Alice Dupont'
    session.commit()

    # Supprimer
    session.delete(user)
    session.commit()

```

=====

RÉCAPITULATIF – CHOISIR LE BON OUTIL

=====

— PILOTES —

OUTIL	BDD	NOTES
psycopg2	PostgreSQL	Standard de facto, C-extension, le plus rapide
psycopg3	PostgreSQL	Nouvelle génération, async natif
asyncpg	PostgreSQL	Async uniquement, ultra-rapide (no DB-API)
pymysql	MySQL/MariaDB	Pur Python, compatible RDS, Azure
mysql-connector-python	MySQL	Officiel Oracle, X DevAPI, requêtes préparées
aiomysql	MySQL	Async (basé sur pymysql)
mysqlclient	MySQL	C-extension, le plus rapide pour MySQL

— MÉTHODES D'INSERTION —

MÉTHODE	VITESSE	USAGE
execute() en boucle	Lente	À éviter
executemany()	Moyenne	MySQL ok, PostgreSQL moyen
execute_values() [psycopg2]	Rapide	PostgreSQL, recommandé
execute_batch() [psycopg2]	Rapide	PostgreSQL, UPDATE/DELETE batch
COPY FROM [psycopg2]	Très rap.	PostgreSQL, import massif
INSERT multi-lignes [MySQL]	Rapide	MySQL, concaténation manuelle

— RÈGLES D'OR —

- ✓ Toujours utiliser des paramètres liés (%s ou %(nom)s) – jamais de f-string
- ✓ Toujours fermer curseur et connexion (with ou finally)
- ✓ PostgreSQL : utiliser psycopg2.sql.Identifier pour les noms de tables/colonnes

- ✓ Utiliser `execute_values` pour INSERT massif sur PostgreSQL
- ✓ `charset='utf8mb4'` pour MySQL (unicode complet, emoji)
- ✓ `connection_timeout` pour éviter les blocages infinis
- ✓ Utiliser un pool de connexions en production (`ThreadedConnectionPool` ou `SQLAlchemy`)
- ✓ Gérer les erreurs de sérialisation (40001) avec `retry` pour `SERIALIZABLE`
- x Ne jamais concaténer du SQL avec des données utilisateur
- x Ne pas utiliser `REPLACE INTO` (supprime + insère, casse les FK)
- x Ne pas oublier `conn.rollback()` dans le `except` (sinon transaction bloquée)
- x Ne pas utiliser `autocommit=True` avec des transactions multi-requêtes

— PARAMÈTRES PAR TYPE DE DRIVER —

DRIVER	PLACEHOLDER	STYLE DICT	EXEMPLE
psycopg2	%s	%(nom)s	<code>execute("...%s...", (val,))</code>
pymysql	%s	%(nom)s	<code>execute("...%s...", (val,))</code>
mysql-connector	%s	%(nom)s	<code>execute("...%s...", (val,))</code>
sqlite3	?	:nom	<code>execute("...?", (val,))</code>
SQLAlchemy	:nom	:nom	<code>execute(text("...:n..."), {'n': v})</code>

=====